VC Generation

References And Borrowing

Goal

Extend Tiny TAC with references without adding a new VCGen core.

Plan:

- model a reference as an explicit value
- lower borrow commands to ordinary assignments, map reads, map writes, and assumes
- run the existing Tiny TAC VCGen

Reference Types

Tiny TAC gains:

$$\tau ::= \text{bool} \mid \text{int} \mid \text{bytemap} \mid \& \ \tau \mid \&\text{mut} \ \tau$$

The main case is a reference to an integer cell:

$$r : \&\text{int} \quad q : \&\text{mut int}$$

Reference Triple

Symbolically, a reference carries:

$$r = \{\text{addr} : i, \text{value} : v, \text{promise} : p\}$$

- `addr`: location pointed to by the reference
- `value`: current value observed through the reference
- `promise`: value that will be committed on release

The `promise` field is unused for constant references, but keeping one shape makes the lowering uniform.

Borrow Commands

```
r := borrow M[i]
r, M2 := borrow_mut M[i]
q := borrow_ref r
q, r2 := borrow_ref_mut r
x := get_ref r
r2 := put_ref r, v
release r
```

Mutable borrowing returns both the reference and a continuation memory. Mutable reborrowing returns the child reference and the resumed parent reference.

Direct Borrow Example

```
entry:
  M := havoc
  i := havoc
  p := borrow M[i]
  x := get_ref p
  release p
  q, M2 := borrow_mut M[i]
  q2 := put_ref q, (x + 1)
  release q2
  ok := M2[i] == x + 1
  assert ok
```

This reads through a constant reference, then updates the same location through a mutable reference.

Lowering: Direct Borrows

Constant borrow:

```
r := borrow M[i]
```

lowers to:

```
r := { addr: i,
       value: M[i],
       promise: havoc }
```

Mutable borrow:

```
r, M2 := borrow_mut M[i]
```

lowers to:

```
r := { addr: i,
       value: M[i],
       promise: havoc }
M2 := M[i := r.promise]
```

Lowering: Use And Release

Read:

```
x := get_ref r
```

lowers to:

```
x := r.value
```

Write:

```
r2 := put_ref r, v
```

lowers to:

```
r2 := { addr: r.addr,
        value: v,
        promise: r.promise }
```

Release:

```
assume r.value == r.promise
```

Mutable Borrow Lowered

```
entry:
  M := havoc
  i := havoc
  r := { addr: i, value: M[i], promise: havoc }
  M2 := M[i := r.promise]
  r2 := { addr: r.addr, value: 7, promise: r.promise }
  assume r2.value == r2.promise
  x := M2[i]
  ok := x == 7
  assert ok
```

Why It Proves

$$\begin{aligned} \mathrm{M2} &= M[i := \mathrm{promise}(r)] \\ \mathrm{value}(\mathrm{r2}) &= 7 \\ \mathrm{promise}(\mathrm{r2}) &= \mathrm{promise}(r) \\ \mathrm{value}(\mathrm{r2}) &= \mathrm{promise}(\mathrm{r2}) \end{aligned}$$

Therefore:

$$\mathrm{promise}(r) = 7$$

So the continuation memory `M2` stores `7` at address `i`.

Mutable Reborrow

```
r, M2 := borrow_mut M[i]
q, r2 := borrow_ref_mut r
q2 := put_ref q, 7
release q2
r3 := put_ref r2, 8
release r3
```

`q` borrows through `r`. After `q` is released, `r2` resumes the parent reference and can overwrite the final promised value.

Reborrow Lowering

```
q := { addr: r.addr, value: r.value, promise: havoc }
r2 := { addr: r.addr, value: q.promise, promise: r.promise }
```

When `q` is released, it updates `r2.value`. When `r2` is released, it updates the original memory promise.

Reborrow Facts

$$\begin{aligned}
\mathrm{M2} &= M[i := \mathrm{promise}(r)] \\
\mathrm{value}(\mathrm{q2}) &= 7 \\
\mathrm{value}(\mathrm{q2}) &= \mathrm{promise}(q) \\
\mathrm{value}(\mathrm{r3}) &= 8 \\
\mathrm{promise}(\mathrm{r3}) &= \mathrm{promise}(\mathrm{r2}) \\
\mathrm{promise}(\mathrm{r2}) &= \mathrm{promise}(r)
\end{aligned}$$

The final release implies `promise(r) = 8`, so `M2[i] == 8`.

Prophecy Variable

The `promise` field is a prophecy-style value:

- chosen nondeterministically at borrow time
- constrained later at release time
- already written into the continuation memory

This is the RustHorn idea adapted to a low-level memory setting: future borrow effects are represented by values in the current formula.

VCGen Boundary

After lowering, VCGen only sees ordinary Tiny TAC:

- assignments
- map reads and stores
- assumptions
- assertions
- havoc values

Borrow well-formedness remains a separate side condition or a separate set of verification obligations.